

TP 2 : DevSecOps Avancé

Pipeline CI/CD Sécurisé - Partie 2

Travail Pratique N°2



Saad KORCHI

Étudiant en Cybersécurité

Encadré par :

Pr. AMAMOU Ahmed

EIDIA Cybersécurité

SOMMAIRE

1	Introduction	3
2	Objectifs pédagogiques - Partie 2	3
3	Pipeline CI/CD (Partie 2)	3
3.1	Fichier .gitlab-ci.yml (security, package, deploy)	3
3.2	Dockerfile	4
4	Démarche Pas à Pas (Partie 2)	5
4.1	Étape 4 : Test de détection de secrets	5
4.2	Étape 5 : Correction et déploiement	5
5	Difficultés et Solutions	5
5.1	Problème 1 : Image Gitleaks	5
5.2	Problème 2 : Tag Node.js invalide	5
5.3	Problème 3 : Image Docker non trouvée	5
5.4	Problème 4 : Dossier uploads manquant	6
6	Conclusion	6
	Captures d'écran	7

1 Introduction

Contexte du travail pratique - Partie 2

Ce rapport présente la seconde partie du Travail Pratique sur l'introduction au **DevSecOps** et aux pipelines **CI/CD sécurisés**. Cette partie couvre l'intégration complète des outils de sécurité, la conteneurisation Docker, le packaging et le déploiement.

2 Objectifs pédagogiques - Partie 2

Les objectifs spécifiques de cette deuxième partie sont :

- Intégrer **Gitleaks** pour la détection de secrets dans le pipeline
- Configurer **npm audit** pour les vulnérabilités des dépendances
- Conteneuriser l'application avec **Docker**
- Automatiser le **packaging** et le **déploiement**
- Comprendre les bonnes pratiques **Shift Left**

3 Pipeline CI/CD (Partie 2)

3.1 Fichier .gitlab-ci.yml (security, package, deploy)

Extrait complet de la configuration des stages security, package et deploy :

```
1 security-secrets-scan:
2   stage: security
3   image:
4     name: zricethezav/gitleaks:latest
5     entrypoint: [""]
6   script:
7     - gitleaks detect --source=. --verbose
8   allow_failure: false
9
10 security-dependencies-scan:
11   stage: security
12   image: node:18
13   script:
14     - npm audit --audit-level=high
15   allow_failure: false
16
17 package-docker:
18   stage: package
19   image: docker:24
20   services:
```

```
21     - docker:24-dind
22 script:
23     - docker build -t $APP_NAME:$APP_VERSION .
24     - docker save $APP_NAME:$APP_VERSION > $APP_NAME.tar
25 artifacts:
26     paths:
27         - $APP_NAME.tar
28     expire_in: 1h
29 only:
30     - main
31
32 deploy-staging:
33     stage: deploy
34     image: docker:24
35     services:
36         - docker:24-dind
37     before_script:
38         - docker load < $APP_NAME.tar
39     script:
40         - docker stop upload-app-staging || true
41         - docker rm upload-app-staging || true
42         - docker run -d --name upload-app-staging -p 3000:3000
43           $APP_NAME:$APP_VERSION
44     environment:
45         name: staging
46         url: http://staging.upload-app.local:3000
47 only:
48     - main
49
50 deploy-production:
51     stage: deploy
52     image: docker:24
53     services:
54         - docker:24-dind
55     before_script:
56         - docker load < $APP_NAME.tar
57     script:
58         - docker stop upload-app-prod || true
59         - docker rm upload-app-prod || true
60         - docker run -d --name upload-app-prod -p 8080:3000 $APP_NAME:
61           $APP_VERSION
62     environment:
63         name: production
64         url: http://www.upload-app.com
65 when: manual
66 only:
67     - main
```

3.2 Dockerfile

Configuration du conteneur Docker :

```

1 FROM node:22.20.0
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm install
5 COPY . .
6 RUN mkdir -p uploads
7 VOLUME ["/app/uploads"]
8 EXPOSE 3000
9 CMD ["node", "server.js"]

```

4 Démarche Pas à Pas (Partie 2)

4.1 Étape 4 : Test de détection de secrets

```

1 echo "const API_KEY = 'sk-test123456789';" > secret.js
2 git add secret.js
3 git commit -m "test secret"
4 git push

```

4.2 Étape 5 : Correction et déploiement

```

1 git rm secret.js
2 git commit -m "fix: suppression du secret"
3 git push

```

5 Difficultés et Solutions

5.1 Problème 1 : Image Gitleaks

Erreur	Error: unknown command "sh" for "gitleaks"
Solution	Ajout de entrypoint: [""] dans la configuration de l'image Gitleaks

5.2 Problème 2 : Tag Node.js invalide

Erreur	node:v22.20.0: not found
Solution	Correction du tag en node:22.20.0 (suppression du "v")

5.3 Problème 3 : Image Docker non trouvée

Erreur	pull access denied for upload-app
Solution	Utilisation de docker save et docker load avec les artifacts GitLab

5.4 Problème 4 : Dossier uploads manquant

Erreur	Error: ENOENT: no such file or directory, open 'uploads/file.txt'
Solution	Ajout de RUN mkdir -p uploads dans le Dockerfile et création du dossier avec .gitkeep

6 Conclusion

Bilan du TP DevSecOps

Ce TP a permis de mettre en place un pipeline DevSecOps complet pour une application Node.js de type upload de fichiers avec :

- **Intégration continue** : Build et test automatisés à chaque commit
- **Sécurité automatisée** : Détection de secrets (Gitleaks) et vulnérabilités (npm audit)
- **Packaging** : Conteneurisation Docker avec volume persistant pour les uploads
- **Déploiement continu** : Staging automatique, production manuelle
- **Shift Left** : Sécurité intégrée dès le début du pipeline

Fin du rapport TP2

Captures d'écran

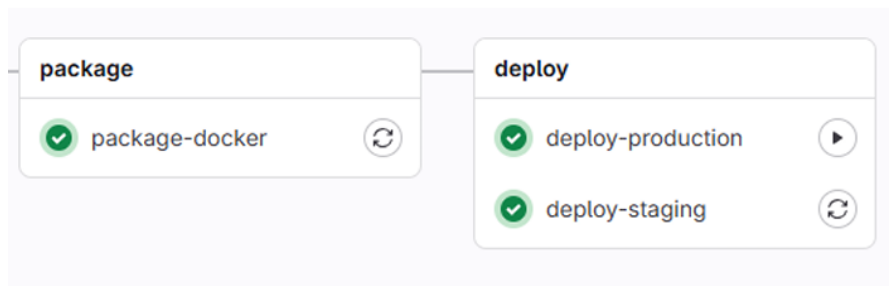


FIGURE 1 – Figure 1

Job Name	Status	Age	Actions
#13095465019: deploy-production	Passed	50:01:37 14 minutes ago	deploy
#13093730291: deploy-production	Passed	50:01:30 1 hour ago	deploy
#13093205231: deploy-production	Passed	50:01:37 2 hours ago	deploy
#13093205226: deploy-staging	Passed	50:01:33 2 hours ago	deploy
#13093205217: package-docker	Passed	50:02:25 2 hours ago	package
#13093205213: security-dependencies-scan	Passed	50:00:35 2 hours ago	security
#13093205208: security-secrets-scan	Passed	50:00:14 2 hours ago	security
#13093205205: test-job	Passed	50:00:36 2 hours ago	test
#13093205196: build-job	Passed	50:00:43 2 hours ago	build

FIGURE 2 – Figure 2

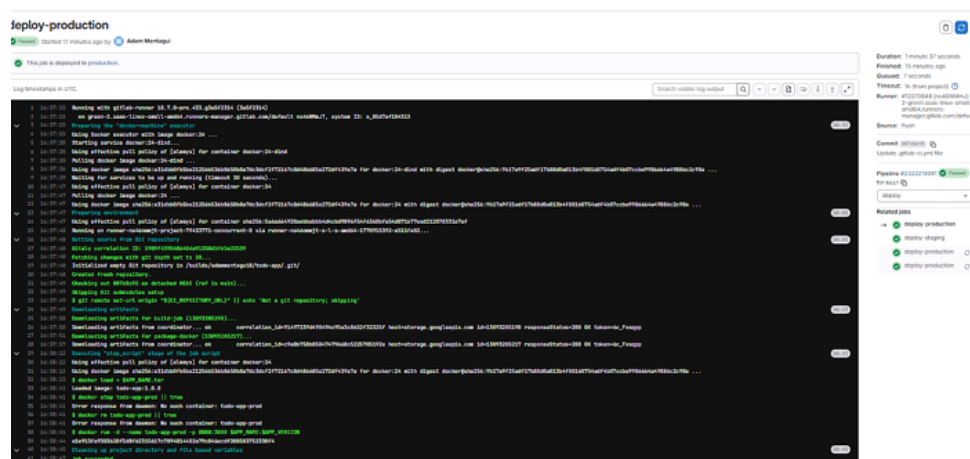


FIGURE 3 – Figure 3

```

172 14:56:17 #7 [3/5] COPY package*.json ./
173 14:56:17 #7 CACHED
174 14:56:17 #8 [4/5] RUN npm install --production
175 14:56:17 #8 CACHED
176 14:56:17 #9 [5/5] COPY . .
177 14:56:17 #9 CACHED
178 14:56:17 #10 exporting to image
179 14:56:17 #10 exporting layers done
180 14:56:17 #10 writing image sha256:e6155e3728f39b2062ca916f77cf01df280ab74dd83b40a0
d65e395720d8f26d done
181 14:56:17 #10 naming to docker.io/library/todo-app:latest done
182 14:56:17 #10 DONE 0.0s
183 14:56:17 $ docker save $APP_NAME:$APP_VERSION > $APP_NAME.tar
184 14:56:43 Uploading artifacts for successful job 00:45
185 14:56:44 Uploading artifacts...
186 14:56:44 todo-app.tar: found 1 matching artifact files and directories
187 14:57:28 Uploading artifacts as "archive" to coordinator... 201 Created correlati
on_id=03f64ff071144a7aaf97be5209038772 id=13093205217 responseStatus=201
Created token=6c_iZstJ6
188 14:57:28 Cleaning up project directory and file based variables 00:01
189 14:57:31 Job succeeded

```

FIGURE 4 – Figure 4



FIGURE 5 – Figure 5